

Practical 7: Queue Implementation using Array and Linked List, with Structure Simulation

→ Practical Aim

To implement queue using array and linked list, and apply queue-based logic to number generation, and structure simulation problems.

Problem 1

A government office has a token counter that issues tokens to visitors. The counter can hold at most n tokens at a time. New visitors join from one end and are served from the other end in the order they arrived. If the counter is full, no new token can be issued, and if it is empty, no one can be served. Given a sequence of join and serve operations, implement this fixed-capacity token system and print the current front token after each operation. Report an error if an operation cannot be performed. Describe how your solution tracks the front and rear of the queue. What happens to your front and rear pointers over time as join and serve operations keep happening, and does your solution handle the case where space appears to run out even though slots have been freed?

Source Code

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/1725b29b455eeb054d70967258fc0b64f9b93c22/PRACTICAL%2007

Output

```
"C:\Users\diyat\OneDrive\Doc  ×  +  v
1. Enqueue (Join)
2. Dequeue (Serve)
3. Exit
Enter choice: 1
Enter token: 23
Token added: 23
Front token: 23

1. Enqueue (Join)
2. Dequeue (Serve)
3. Exit
Enter choice: 1
Enter token: 34
Token added: 34
Front token: 23

1. Enqueue (Join)
2. Dequeue (Serve)
3. Exit
Enter choice: 1
Enter token: 68
Token added: 68
Front token: 23

1. Enqueue (Join)
2. Dequeue (Serve)
3. Exit
Enter choice: 1
Enter token: 89
```

```
1. Enqueue (Join)
2. Dequeue (Serve)
3. Exit
Enter choice: 1
Enter token: 89
Token added: 89
Front token: 23

1. Enqueue (Join)
2. Dequeue (Serve)
3. Exit
Enter choice: 2
Token served: 23
Front token: 34

1. Enqueue (Join)
2. Dequeue (Serve)
3. Exit
Enter choice: 2
Token served: 34
Front token: 68
```

Conclusion

The circular queue follows FIFO order and reuses empty spaces efficiently. It performs enqueue and dequeue operations using front and rear pointers.

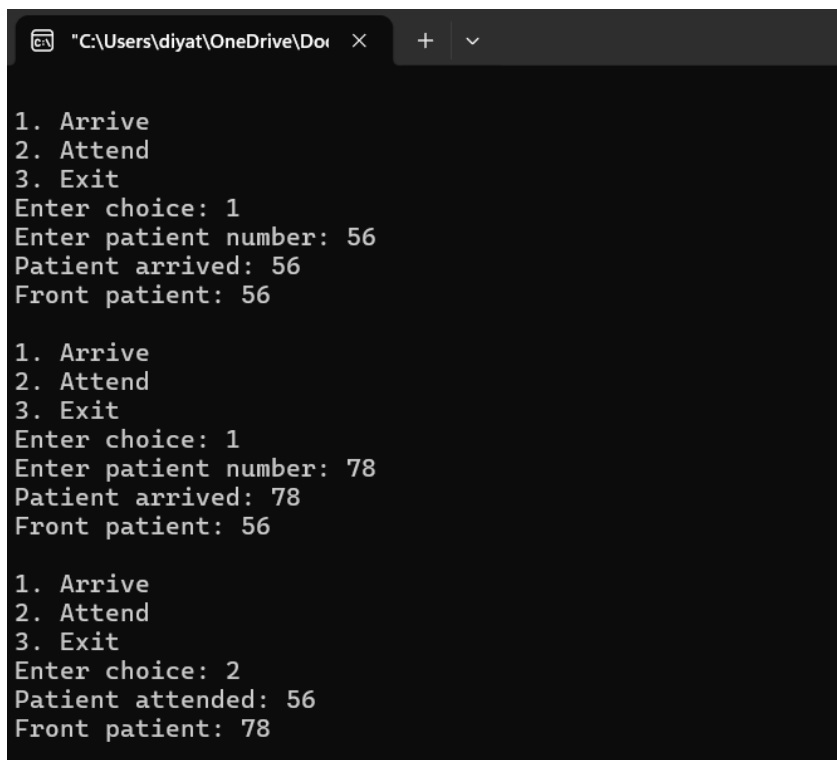
Problem 2

A hospital emergency ward receives patients continuously and attends to them strictly in the order they arrived, with no upper limit on how many can be waiting at once. New patients are added at the back and the doctor always attends to the patient at the front. Given a sequence of arrive and attend operations, implement this unlimited patient queue and print the current front patient after each operation. Describe how your solution differs from Problem 1 in how it manages capacity. What happens in your solution when the attend operation is called on an empty ward with no patients waiting ?

Source code

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/1725b29b455eeb054d70967258fc0b64f9b93c22/PRACTICAL%207

Output



```
"C:\Users\diyat\OneDrive\Docu... X + v
1. Arrive
2. Attend
3. Exit
Enter choice: 1
Enter patient number: 56
Patient arrived: 56
Front patient: 56

1. Arrive
2. Attend
3. Exit
Enter choice: 1
Enter patient number: 78
Patient arrived: 78
Front patient: 56

1. Arrive
2. Attend
3. Exit
Enter choice: 2
Patient attended: 56
Front patient: 78
```

Conclusion

The circular queue follows FIFO order and can grow as new patients arrive. It does not have a fixed size and handles empty queue conditions properly.

Challenges Faced

1. Problem Faced : Managing front and rear pointers when the queue becomes full or empty was confusing.

Solution: Used circular movement with $(\text{rear} + 1) \% 5$ and $(\text{front} + 1) \% 5$ to reuse empty spaces.

2. Problem faced : Handling the queue when there were no patients was confusing.

Solution: Checked `front == NULL` before performing dequeue and displayed an error message.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] A simple array queue eventually reports being full even when slots have been freed at the front. Explain why this happens structurally — not by tracing an example, but by describing what the front and rear pointers represent and what their movement pattern causes over time.

Solution: A simple array queue, the front moves forward when elements are removed, but the rear also keeps moving forward when new elements are added. The rear does not move back to use the empty spaces created at the front. So, after reaching the last array position, the queue may report full even though some earlier positions are empty. A circular queue solves this by allowing front and rear to wrap around.

2. [Problem 2] Your linked list queue has no capacity limit. If arrive operations vastly outnumber attend operations over a long period, what real consequence would this have on the system, and where in your code is that consequence invisible?

Solution: If arrive operations are much more than attend operations, more and more nodes will be created. This can use a large amount of memory and may eventually slow down or crash the system if memory is exhausted. This consequence is invisible in the code because there is no maximum size or memory-limit check; new Node() keeps creating nodes whenever a patient arrives.